绪论

基本概念和术语

- 数据 —— 能够输入到计算机中并且被计算机程序识别和处理的符号的集合
- 数据元素 —— 数据的基本单位
- 数据对象 —— 相同性质的数据元素的集合
- 数据类型
  - 原子类型 —— 不可再分
  - 结构类型 —— 可再分
  - 抽象数据类型 —— 数据对象+数据关系+操作
- 数据结构 —— 是相互之间存在一种或多种特定关系的数据元素的集合

4.9元包邮@5分钱/每张

微信刷题小程序:酷酷刷题
免费刷肖1000余海量题等18位名师题库
支持错题导出、添加笔记、电脑端刷题...

数据结构的三要素

1. 逻辑结构
   - 线性
     - 线性表
     - 栈（受限线性表）
     - 队列（受限线性表）
     - —— 一对一
   - 非线性
     - 集合
     - 树形结构 —— 一对多的关系
     - 图状结构 —— 多对多的关系
2. 存储结构（物理结构）
   1. **顺序存储** —— **逻辑上相邻，物理位置上也相邻**
      - **优点：实现随机存取**
      - **缺点：占用一整块存储单元**
   2. **链式存储** —— **逻辑上相邻，物理位置上不一定相邻**
      - **优点：充分利用存储单元**
      - **缺点：元素存储指针需要占用额外的存储空间， 且只能实现顺序存取**
   3. 索引存储 —— 存储信息的同时，还建立附加的索引表
      - **优点：检索速度快**
      - **缺点：增加索引表，占用存储空间**
   4. 散列存储 —— Hash存储，通过哈希函数计算元素存储地址
      - **优点：检索、增加、删除节点比较快**
      - **缺点：散列函数选取不好，造成元素冲突**
3. 数据的运算
   - 定义 —— 逻辑结构 —— 运算功能
   - 实现 —— 存储结构 —— 运算操作步骤

算法与算法分析

- 算法 —— 是对特定问题求解步骤的一种描述
- 算法特性
  1. 输入 —— 0个或多个输入
  2. 输出 —— 一个或多个输出
  3. 有穷性 —— 一个算法在执行有穷步后的结束
  4. 确定性 —— 每条指令必须有确切的含义
  5. 可行性 —— 算法的操作可以实现
- 算法效率的度量
  1. 时间复杂度
     1. 最好时间复杂度 —— 指在最好的情况下，算法的时间复杂度
     2. 最坏时间复杂度 —— 指在最坏的情况下，算法的时间复杂度
     3. 平均时间复杂度 —— 指所有可能输入实例在等概率出现的情况下，算法的期望运行时间
     4. **常见的渐进时间复杂度比较** —— $O(1) < O(\log_2 n) < O(n) < O(n\log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$
  2. 空间复杂度
     - 它是问题规模n的函数
     - 算法原地工作是指算法所需的辅助空间为常量O(1)

配套课程公众号[有机研]
永久联系微信:4550060

线性表

线性表的定义及基本操作

- 定义 —— 具有相同数据类型的**n个数据元素的有限序列**
- 基本操作
  1. Initlist(&L)：初始化一个空的线性表
  2. Length(L)：求表长，返回元素个数
  3. LocateElem(L,e)：按值查找
  4. GetElem(L,i)：按位查找
  5. ListInsert(&L,i,e)：插入操作
  6. Empty(L)：判空操作。 若L为空返回True,否则返回false
  7. DestroyList()：销毁操作。 销毁线性表，释放存储空间

线性表的存储结构

- 定义 —— **顺序表是线性表的顺序存储（物理结构）是一种随机存取的存储结构**
- 顺序表上的基本操作 —— **顺序表的初始化**
  - 静态分配
  - **动态分配　L.data = (ElemType*)malloc(sizeof(ElemType)*InitSize)**
- 插入操作
  - **在顺序表L的第i个位置插入新的元素**
  1. 判断 i 的位置是否有效 —— if(i< 1 || i > L.length+1)　return false
  2. 判断存储空间是否已满 —— if(L.length >= MaxSize)　return false
  3. 将第 i 个元素及之后的元素后移
     - for(int j = L.length; j >= i; j--)
     - L.data[j] = L.data[j-1]　**元素后移**
  4. 插入新元素 —— L.data[i-1] = e
  5. 表长 + 1 —— L.length++
  - **平均时间复杂度为O(n)**
- 删除操作
  - **删除顺序表L中的第 i 个位置的元素**
  1. 判断 i 的位置是否有效 —— if(i< 1 || i > L.length+1)　return false
  2. 将被删除的元素赋值给e —— e = L.data[i-1]
  3. 将第 i 个位置后的元素前移
     - for(int j = i ; j <= L.length; j++)
     - L.data[j-1] = L.data[j]　**元素前移**
  4. 删除元素，线性表长 -1 —— L.length--
  - **平均时间复杂度为O(n)**
- 按值查找（顺序查找）
  - 在顺序表L中查找第一个等于 e 的元素
  - 循环查找
    - for(i = 0; i<L.length; i++)
    - if(L.data[i] == e)
    - return i+1 —— **下标为i的元素值等于e，返回其次序i+1**
  - **平均时间复杂度为O(n)**

配套课程公众号[有机研]
永久联系微信:4550060

线性表的链式表示

单链表

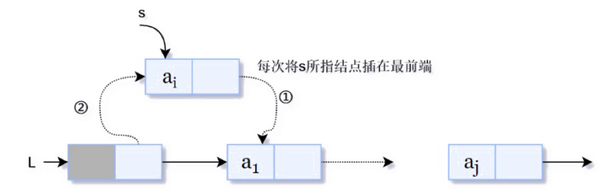
单链表的定义

- 定义 —— 线性表的链式存储方式
- 非随机存取
- 查找结点时从表头开始遍历
- **头节点与头指针的区分**
  - 不管带不带头结点，头指针始终指向链表的第一个结点
  - 头结点是带头节点时链表的第一个结点
    - **优点：统一操作，无须特殊处理**
    - **优点：空表和非空表的操作得到统一**

建立单链表

头插法（逆序）

方法：将新结点插入到当前链表的表头，即头结点之后



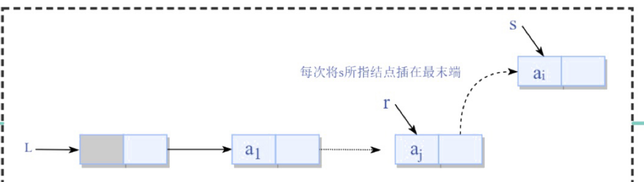
- **思想：逆序插入到链表中，有一个头指针，每次新结点在表头**

1. **s = (LNode*)malloc(sizeof(LNode))　//创建新结点**
2. **s->data = x**
3. **s->next = L->next　　//新结点的后继指向第一个结点**
4. **L->next = s　//头节点指向新节点**

时间复杂度：平均时间复杂度为O(n)

尾插法（顺序）

方法：将新结点插入到当前链表的表尾



- **思想：加入尾指针r用来指向链表的尾结点**

初试状态头指针L=NULL, 尾指针r = NULL

将新结点插入到尾指针r所指节点后面，尾指针r指向新结点

1. **s = (LNode*)malloc(sizeof(LNode))　//创建新结点**
2. **s->data = x**
3. **r->next = s　　//r的后继指针指向新结点**
4. **r = s　　//r指向新的表尾结点**

时间复杂度：平均时间复杂度为O(n)

单链表判空条件

- **有头结点　L->next = null**
- **无头结点　L = null**

考点

结点插入操作

思想：1.检查位置的合法性

2.查找表中第 i-1给结点，即前驱结点，然后删除

1. p = Getelem(L, i-1)　查找插入位置的前驱结点
2. s->next = p->next　新结点的指针s指向p的后继结点
3. p->next = s　p指针指向新插入的结点

- **性能分析　插入新结点仅为O(1)，查找平均时间复杂度O(n)**

删除结点操作

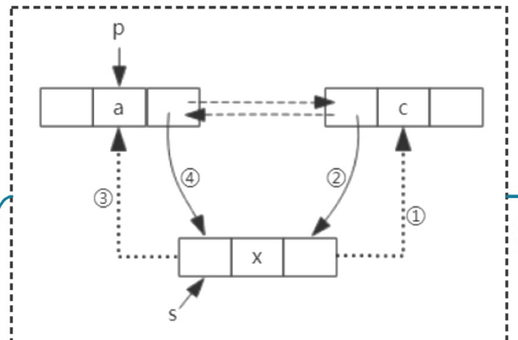
思想：1.先检查位置的合法性

2.查找删除位置的前驱结点

1. p = Getelem(L, i-1)　查找删除位置的前驱结点
2. q = p->next　令q指向被删除结点
3. p->next = q->next　将*q结点从链表中断开
4. Free(q)　释放结点的存储空间

双链表

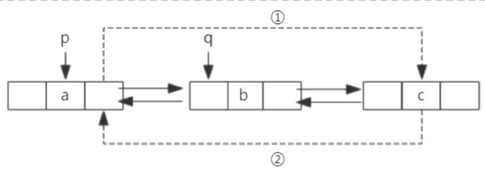
- 定义 —— 双链表有两个指针（prior 指向前驱结点，next指向后继结点）
- 性能分析 —— 双链表的插入、删除的时间复杂度均为O(1)
- 双链表的操作



插入操作

1. **s->next = p->next　//将要插入结点的后继指针指向后继结点**
2. **p->next ->prior = s　//将后继结点的前驱指针指向新结点**
3. **s->prior = p　//将新结点的前驱指针指向前驱结点**
4. **p->next = s　//将前驱结点的指针指向新结点**

不能更变顺序，步骤①②必须在③④之前



删除操作

1. **p->next = q->next　//将要删除的结点的前驱结点的后继指针指向要删除结点的后继指针**
2. **q->next->prior = p　//将要删除结点的后继结点的前驱指针指向删除结点的前驱结点**
3. **free(q)　//释放结点内存**

循环链表

循环单链表

表中最后一个结点的指针不是null, 而是指向头结点，从而整个链表形成环

- **循环判空条件** —— 1. **r->next = L　//尾指针是否指向头结点**
- 性能分析
  - 设头指针对表尾的操作　O(n)
  - 设尾指针对表头，表尾的操作　O(1)

循环双链表

在循环单链表中加入头结点prior指针指向表尾结点

- **循环判空条件** —— 1. **L->next = L && L->prior = L　//头结点的prior域和next域都等于L**

配套课程公众号[有机研]
永久联系微信:4550066

栈

栈的定义及操作

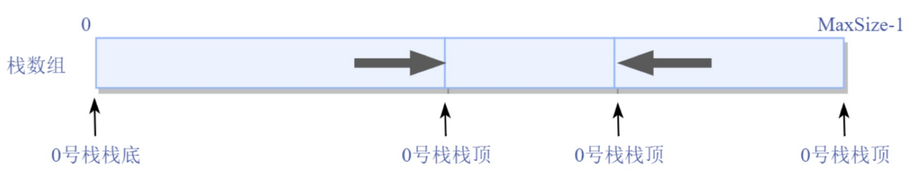
- 定义：**栈（Stack）只允许在一端进行插入或删除操作的线性表**
- 特点：
  - **先进后出，后进先出**
  - 栈顶：允许插入删除的一端
  - 栈底：不允许插入删除的一段
  - 空栈：不含任何元素的空表
- 栈的数学性质：**n个不同元素进栈，出栈元素不同的排列个数为** $\frac{1}{n+1}C_{2n}^{n}$
  - 重点记住，做选择题使用
- 基本操作：
  1. InitStack(&s)：初始化一个空栈
  2. StackEmpty(S)：判断栈是否为空，返回布尔值
  3. Push(&S,x)：进栈，把元素x压入栈顶
  4. Pop(&s,&x)：出栈，元素x弹出栈顶
  5. GetTop(S,&x)：读取栈顶元素，用x返回栈顶元素

栈的顺序存储结构

- 顺序栈的实现：
  - 利用一组地址连续的存储单元存放自栈底到栈顶的数据元素
  - 附设一个指针(top)指向当前栈顶的位置

```
Typedef strcuk {
    ElemType data[Maxsize];   //存放栈顶元素
    int top;    //栈顶指针top
} SqStack;
```

- 顺序栈的基本操作：
  - **初始化： S.top = -1；栈顶元素：S.data[S.top]**
  - **有部分教辅会将栈指针S.top定义为0，相当于规定top指向栈顶元素的下一个存储单元**
  - **栈空条件: S.top == -1**
  - **栈满条件：S.top = MaxSize-1**
  - **栈长：S.top + 1**
  - 进栈操作：
    - 栈不满时，栈顶指针S.top先加1，再送元素x到栈顶元素
    - **S.data [++S.top] = x; //指针加1，再进栈**
  - 出栈操作：
    - 栈非空时，先取栈顶元素值x，再将栈顶指针S.top先减1
    - **x = S.data[S.top --]; //先把元素出栈，指针再-1**
  - 读取栈顶元素：**x = S.top[S.top]**
- 共享栈：

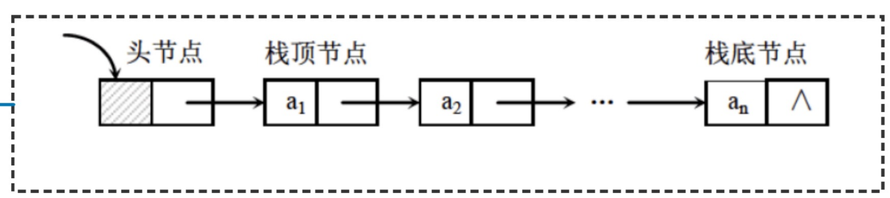


两个顺序栈共享存储空间

  - 两个顺序栈共享存储空间，两个栈顶向中间延伸
  - **top0 = -1时0号栈为空， top1 = Maxsize时1号栈为空**
  - **栈满条件：top1 - top0 = 1**
  - 共享栈是为了更有效地利用存储空间，两个栈空间相互调节，只有在整个存储空间被占满时才发生上溢。
  - **存取数据的时间复杂度为O(1)**

栈的链式存储结构

- 链栈基本结构：



- 采用链式存储结构的栈称为链栈
  - **优点：便于多个栈共享存储空间和提高效率，不会出现栈满上溢的情况**
  - **通常采用单链表实现。**采用链式存储，便于结点的插入与删除基本操作和链表类似，入栈和出栈的操作都在表头进行

配套课程公众号[有机研]
永久联系微信:4550060

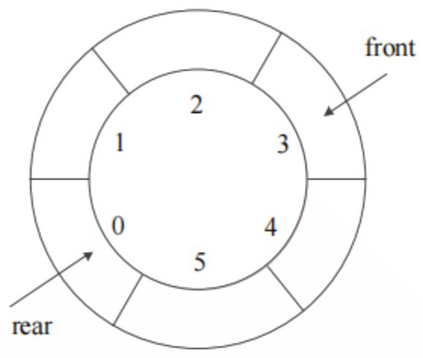
队列

队列的定义及基本操作

- 定义
  - 队列（Queue）只允许在表的一端进行插入，而在表的另一端进行删除
  - 一种操作受限的线性表
  - 操作特性：先进先出(FIFO)
- 队列的基本操作
  1. InitQueue(&Q)：初始化队列，构造一个空队列
  2. QueueEmpty(Q)：判队列空，若队列 为空返回 true ， 否则返回 false
  3. EnQueue(&Q,&x)：入队，若队列未满，将x加入，使之成为新队尾
  4. DeQueue(&Q,&x)：出队，若队列非空，删除对头元素，返回x
  5. GetHead(Q &x) ： 读队头元素 若队列 非空，则将队头元素赋值给 X

队列的顺序存储结构

- 队列的顺序存储
  - 思想
    - 设置两个指针
      - front 指向对头元素
      - rear 指向队尾元素的下一个位置
  - 操作
    - 入队操作：　队不满时，先送至队尾元素，再将队尾指针 +1
    - 出队操作：　队不空时，先取对头元素值，再将队头指针 +1
    - **初试状态（队空条件）：Q.front == Q.rear == 0**
    - **队长：rear - front**
    - 假上溢：不能用rear = MaxSize 判断队满
- 循环队列



思想：将顺序队列臆造成一个环状的空间，把存储元素的表在逻辑上看成环

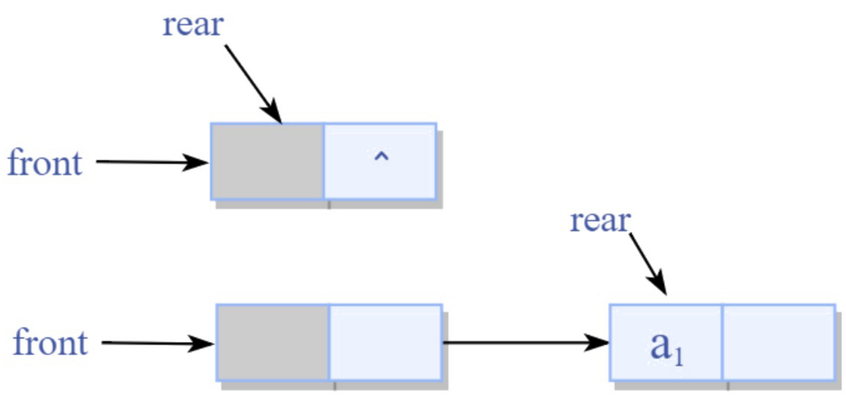
- **初始化：Q.front = Q.rear = 0**
- **队首指针进 Q. front= (Q.rear+1)% MaxSize**
- **队尾指针进 : Q.rear= (Q.rear+1)% MaxSize**
- **队满条件：Q.rear= (Q.rear+1)% MaxSize= front**
- **队空条件仍： Q.front==Q rear**
- **队列元素个数：(Q.rear + MaxSize - Q.front)%MaxSize**
- 操作
  - 入队
    - if（Q.rear +1 % MaxSize == Q.front ） return false　//判断是否队满
    - **Q.data[Q.rear] = x;　//向队尾插入x**
    - **Q.rear = (Q.rear +1) % MaxSize;　//队尾指针 +1 取模**
  - 出队
    - if（Q.rear == Q.front） return false;　//判断是否队空
    - **x = Q.data[Q.front];　//元素出队**
    - **Q.front = (Q.front +1) % MaxSize;　//对头指针加1取模**

队列的链式存储结构

- 链式存储

不带头结点的链式存储

缺点：不带头结点的操作比较麻烦



带头结点的链式存储

思想：实际上是一个同时带有对头指针和队尾指针的单链表

- 头指针指向头结点
- 尾指针指向尾结点
- **优点**
  - **带头结点，插入和删除的操作得到统一**
  - **且单链表表示的队列特别适合数据元素变得比较大情形**

- 链式队列的基本操作
  - **入队**
    - 建立一个新结点，将新结点插入链表的队尾，并让rear指针指向新结点
    - **s->data = x; s->next = null;　//创建新结点，插入到队尾**
    - **Q.rear->next = s;　//将rear指针所指的结点的后继指针指向新结点**
    - **Q.rear = s;　//rear指针后移到新结点**
  - **出队**
    - 首先判断是否为空，若不空则队头元素出队，将其从链表中删除；并让front指向下一个结点
    - **若队列中只有一个结点，则令 Q.rear == null && Q.front == null;**
    - **if（Q.front == Q.rear）　return false;　//空队**
    - **LinkNode *p = Q.front->next;**
      - **p是指向被删除结点的指针**
      - **被删除结点是头结点之后的结点**
    - **x = p->next;　//被删除的元素赋值给x**
    - **Q.front->next = p->next;　//头结点的后继指针指向被删除结点的后面的结点**
    - **if（Q.rear == p）**
    - **Q.rear = Q.front;　//若原队列只有一个元素，删除后队列为空**
    - free(p)

双端队列

- 基础概念
  - 允许两端都可以进行入队和出队操作的队列
    - 输出受限　允许一端插入删除，另一端只允许插入
    - 输入受限　允许一端插入删除，另一端只允许删除

配套课程公众号[有机研]
永久联系微信:455000600

串、数组和特殊矩阵

数组

数组的基本概念

数组是由n(n≥1)个相同类型的数组元素构成的有限序列，每个数组元素称为一个数组元素，每个元素在n个线性关系中的序号称为该元素的下标。

多维数组的两种映射方式

- 按行优先
  - 基本思想：先行后列，先存储行号较小的元素，行号相等先存储列号较小的元素
  - $LOC(a_{i,j}) = LOC(a_{0,0}) + [i \times （h_2+1） + j] \times L$
- 按列优先
  - $LOC(a_{i,j}) = LOC(a_{0,0}) + [j \times （h_2+1） + i] \times L$

特殊矩阵的压缩存储

- 压缩存储 —— 指为多个值相同的元素只分配一个存储空间，对零元素不分配存储空间，目的是节省存储空间
- 特殊矩阵 —— 具有许多相同矩阵元素或零元素，并且这些相同矩阵元素或零元素的分布具有一定的规律性。例如对称矩阵、上（下）三角矩阵、对角矩阵
- 特殊矩阵的压缩存储方法 —— 找出特殊矩阵中值相同的矩阵元素的分布规律，把那些呈现规律特性分布的、值相同的多个矩阵元素存储到一个存储空间中

稀疏矩阵及其存储方式

- 矩阵中**非零元素的个数t**，相对矩阵元素个数s来说非常少，**即s>>t的矩阵为稀疏矩阵。**例如一个100×100的矩阵，该矩阵中只有少于100个非零元素
- 采用常规的方法存储稀疏矩阵，相当浪费存储空间，因此仅仅存储非零元素

$$A_{6,7}=\begin{bmatrix}0&0&1&0&0&0&0\\0&2&0&0&0&0&0\\3&0&0&0&0&0&0\\0&0&0&5&0&0&0\\0&0&0&0&6&0&0\\0&0&0&0&0&7&4\end{bmatrix}$$

i	j	$A_{i,j}$
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4

稀疏矩阵A对应的三元组表示

- 稀疏矩阵的三元组的存储方式
  - 数组存储
  - 十字链表法存储

串

串的定义

概念：串是由零个或多个字符组成的有限序列

- S = ‘a1a2a3...an’(n≥0)
  - ai可以是字母，数字或其他字符
  - n为长度，n=0表示空串
- **子串的位置是指其在主串中以子串的第一个字符位置表示**

例如：有串 A = ‘China Beijing’，B = 'Beijing', C = 'China’，则它们的长度分别为 13,7和5。
B和C是A的子串，B在A中的位置是7，C在A中的位置是1

- **有一个或多个空格（空格是特殊字符）组成的串成为空串，它不是空串，长度为空格字符的个数**

串的存储结构

- 定长顺序存储表示
  - 类似于线性表顺序存储
  - 用一组连续地址的存储单元存储串值的字符序列，为每个串变量分配一个固定长度存储区，即定长数组
  - 串实际长度<=MAXLEN,超出部分被截断

```
#define MAXLEN 255      //预定义最大串长为255
typedef strcut{
    char cha[MAXLEN];   //每个分量存储一个字符
    int length;         //串的实际长度
}SString;
```

定长顺序存储表示

- 堆分配存储表示 —— 仍然以一组地址连续的存储单元存放串值的字符序列，但其存储空间在程序执行过程中动态分配得到的
- 块链存储表示 —— 类似于线性表的链式存储结构，每个结点可以存放一个字符也可以存放多个字符

串的基本操作

- StrAssign(&T,chars) —— 赋值操作，把串T赋值为chars
- StrCopy(&T,S) —— 复制操作。由串S复制得到串T
- StrCompare(S,T) —— 比较操作，S>T,返回值大于0；S=T，返回值=0；S<T返回值小于0
- StrLength(S) —— 求串长，返回串s的元素个数
- SubString(&sub,S,pos,len) —— 求子串，用sub返回串S的第pos个字符起长度为len的字串
- Concat(&T,S1,S2) —— 串连接，用T返回S1和S2连接而成的新串

串的普通模式匹配

- 设有两个串s和t，串t的定位就是要在串s中找到一个与t相等的子串。通常把s称为**目标串，**把t称为**模式串**，因此串定位查找也称为**模式匹配。**
- 从主串的第一个位置开始，与子串的第一个位置比较，若相等，则继续逐个比较后序字符；否则从主串的下一个位置比较
- 最坏时间复杂度O（n*m） —— n和m分别为主串和子串的长度

KMP算法

- 定义
  - 前缀：指除了最后一个字符以外，字符串的所有头部子串
  - 后缀：除了第一个字符外，字符串的所有尾部子串
  - 部分匹配值：字符串的前缀和后缀重的最长相等前后缀长度
  - **大致思想：就是主串的指针不需要移动，只需要移动子串的指针即可**
- Next数组
  - **求出部分匹配值数组，右移动一位，（最左补-1）整体+1**
  - next函数的公式：next[j]

$$\text{next}[j]=\begin{cases}-1 & \text{当 } j=0 \text{ 时}\\ \text{MAX}\{k \mid 0<k<j \text{ 且 } "t_0t_1\cdots t_{k-1}"="t_{j-k}t_{j-k+1}\cdots t_{j-1}"\} & \text{当此集合非空时}\\ 0 & \text{其他情况}\end{cases}$$

配套课程公众号[有机试]
永久联系微信:4550060

树与二叉树（1）

树的基本概念

- 树是n(n>=0)个结点的有限集合。当n=0时，称为空树。
- 树的定义是递归的，是一种递归的数据结构
- 树适合表示具有层次结构的数据。树是一种逻辑结构同时也是一种分层结构。
- 特点
  - 树的根结点没有前驱除根节点外，所有结点有且只有一个前驱结点
  - 树中所有结点有零个或多个后继结点

基本术语

结点

- 根结点 —— 树只有一个根结点
- 结点的度 —— **一个结点的子节点的个数**
  - **度为0　叶子结点（终端结点）**
  - **度不为0　分支结点（非终端节点）**
- 结点关系
  1. 祖先结点
  2. 子孙结点
  3. 双亲结点
  4. 孩子结点
  5. 兄弟结点
- **结点的度，高度层次**
  - **层次　从树根开始，根结点为第一层**
  - **深度　从根结点开始自顶向下逐层累加**
  - **高度　从叶子结点开始自底向上逐层累加**

树

- 树的度　树中结点最大的度数
- 树的高度/深度　树中结点的最大层次
- 树的性质
  1. **树中的结点数等于所有结点的度数加1**
  2. **度为m的树中第 i 层上至多有 m^{i-1}个结点（i ≥1）**
  3. **高度为 h 的 m 叉树至多有$(m^h-1)/(m-1)$个结点**
  4. **具有 n 个结点的 m 叉树的最小高度为$\lceil \log_m(n(m-1)+1) \rceil$**

配套课程公众号[有机研]
永久联系微信:4550060

树与二叉树（2）

二叉树的概念

二叉树的定义及特点

二叉树的定义

每个结点至多有两棵子树（即二叉树中不存在度大于2的结点），并且二叉树的结点有左右之分

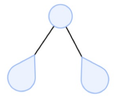
二叉树的五种形态

空二叉树

只有一个根结点

只有左子树

只有右子树



左右子树都有

特殊二叉树

1. **满二叉树** —— 除叶子结点外，每个结点度数均为2
2. **完全二叉树**
   - 与其对应满二叉树中编号1~n 一一对应
   - 特点 —— **度为1的结点只能有一个，且该结点只有左孩子没右孩子**
3. **二叉排序树** —— **左子树的关键字 < 根 < 右子树关键字**
4. **平衡二叉树** —— **树上任意结点的左子树和右子树的深度之差不超过1**

注意：二叉树与度为2的有序树的区别

- 二叉树可以为空，而度为2的树至少有3个结点
- 二叉树的结点次序是有序的，而度为2的树左右次序是相对而言的，若某结点只有一个孩子结点，则无需区分左右

二叉树的性质

1. **非空二叉树上的叶子结点数 = 度为2的结点数+1　　重要公式：$n_0 = n_2 + 1$**

非空二叉树上第k层上至多有 2^{k-1} 个结点（k ≧ 1）。

高度为h的二叉树至多有 $2^h - 1$ 个结点（h ≧ 1）。

对于完全二叉树按从上到下、从左到右的顺序一次编号 1，2，…，n，则有以下关系：

1. 当 i >1时，结点 i 的双亲的编号为 ⌊i/2⌋，即当 i 为偶数时，其双亲的编号为 i/2，它是双亲的左孩子；当 i 为奇数时，其双亲的编号为(i-1)/2，它是双亲的右孩子
2. 当 2i ≦ n 时，结点 i 的左孩子编号为 2i，否则无左孩子
3. 当 2i+1 ≦ n时，结点 i 的右孩子编号为 2i+1，否则无右孩子
4. 结点 i 所在层次（深度）为 $\lfloor \log_2 i \rfloor + 1$

具有 n 个（n > 0）结点的完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$。

完全二叉树

- 若 i ≦ ⌊n/2⌋，则结点 i 为分支结点，否则为叶子结点
- 叶子结点只可能在层次最大的两层出现。对于最大层次中的叶子结点，都一次排列在该层最左边的位置上
- 若有度为1的结点，则只能有一个，且该结点只有左孩子而无右孩子（重要特征）
- 按层序编号后，一旦出现某结点（编号为 i ）为叶子结点或只有左孩子，则编号大于 i 的结点均为叶子结点
- 若 n 为奇数，则每个分支结点都有左孩子和右孩子；
  若 n 为偶数，则编号最大的分支结点（编号为 n/2）只有左孩子，
  没有右孩子，其余分支结点左、右孩子都有

配套课程公众号[有机研]
永久联系微信:4550060

二叉树的遍历（递归方式）

递归

1 先序遍历

```
void PreOrder(BiTree T)
{
    if(T!=NULL)
    {
        visit(T);
        PreOrder(T->lchild);
        PreOrder(T->rchlid);
    }
}
```

1. 访问根结点
2. 先序遍历左子树
3. 先序遍历又子树

根左右

若二叉树为空，则什么也不做，否则

2 中序遍历

```
void InOrder(BiTree T)
{
    if(T!=NULL)
    {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchlid);
    }
}
```

1. 中序遍历左子树
2. 访问根结点
3. 中序遍历右子树

左根右

若二叉树为空，则什么也不做，否则

3 后续遍历

```
void PostOrder(BiTree T)
{
    if(T!=NULL)
    {
        PostOrder(T->lchild);
        PostOrder(T->rchlid);
        visit(T);
    }
}
```

1. 后序遍历左子树
2. 后续遍历右子树
3. 访问根结点

左右根

若二叉树为空，则什么也不做，否则

配套课程公众号[有机研]
永久联系微信:4550060

二叉树的遍历（非递归方式及层次遍历）

- 非递归
  - 1 先序遍历

```
void PostOrder(BiTree T)
{
    Inistack(S);
    P = T;
    while(P || !StackEmpty(S))
    {
        if(P)
        {
            printf(p->data);
            push(S,P);
            P = P->lchild;
        }
        else
        {
            pop(S,P);
        }
    }

}
```

  - 2 中序遍历

```
void InOrder(BiTree T)
{
    Inistack(S);
    P = T;
    while(P||!StackEmpty(S))
    {
        if(P)
        {
            push(S,P);
            P = P->lchild;
        }
        else
        {
            pop(S,P);
            printf(p->data);
            P = P->rchlid;
        }
    }
}
```

  - 3 后续遍历

```
void PostOrder(BiTree T)
{
    Inistack(S);
    P = T;
    while(P||!StackEmpty(S))
    {
        if(P)
        {
            push(S,P);
            P = P->lchild;
        }
        else
        {
            pop(S,P);
            if(P->rchild && (P->rchild).tag==0)
            {
                push(S,P);
                P = P->rchlid;
            }
            else
            {
                printf(p->data);
                P.tag = 1;
                P = NULL;
            }
        }
    }
}
```

- 层次遍历

```
void LevelOrder(BiTree T)
{
    InitQueue(Q);     //初始化辅助队列
    BiTree p;
    EnQueue(Q,T);   //根结点入队
    while(!IsEmpty(Q))  //队列不空循环
    {
        DeQueue(Q,p);  //队头元素出队
        visit(p);       //访问当前p所指向结点
        if(p->lchild!=NULL)
          EnQueue(Q,p->lchild);   //若左子树不空，则左子树入队
        if(p->rchild!=NULL)
          EnQueue(Q,p->rchlid);  //若右子树不空，则右子树入队
    }
}
```

1. 借助一个辅助队列，先将二叉树根节点入队，然后出队，访问该结点
2. 若它有左子树，则左子树根结点入队
3. 若它有右子树，则将右子树根结点入队
4. 然后出队，对出队结点访问，如此反复，直到队列为空

配套课程公众号[有机研]
永久联系微信:4550060

线索二叉树

线索二叉树

基础概念

二叉树的线索化是指将二叉链的空指针改为指向前驱或后继的线索

若无左子树，令lchilid指向其前驱结点；若无又子树，令rchlid指向其后继结点

增加两个标志域表面当前指针域所指对象是左右结点还是直接前驱或后继

线索二叉树标志域

- ltag
  1. **0，lchild指向结点的左孩子**
  2. **1，lchild指向结点的前驱**
- rtag
  2. **0，rchild指向结点的右孩子**
  3. **1，rchild指向结点的后继**

lchild	LTag	data	RTag	rchild

线索二叉树的构造

附设指针 pre 指向刚刚访问过的结点 ， 指针指向正在访问的结点 ， 即 pre 指向前驱
在中序遍历的过程中，检指针是否为空，若为空就将它指向 pre ：
检查 pre 的右指针是否为空，若为 就将它指 ，

中序遍历对二叉树线索化的递及步骤

1. 如果p非空，左子树递归线索化。
2. 如果p的左孩子为空，则给p加上左线索，将其LTag置为1，让p的左孩子指针指向pre（前驱）；否则将p的LTag置为0。
4. 如果pre的右孩子为空，则给pre加上右线索，将其RTag置为1，让pre的右孩子指针指向p（后继）；否则将pre的RTag置为0。
6. 将pre指向刚访问过的结点p，即pre=p。
6. 右子树递归线索化。

中序遍历对二叉树线索化的递及算法

```
void InThread(ThreadTree &p,ThreadTree &pre)
{
    //中序遍历对二叉树线索化的递归算法
    if(p!=NULL)
    {
        InThread(p->lchild,pre);    //递归，线索化左子树
        if(p->lchild == NULL)       //左子树为空，建立前驱线索
        {
            p->lchild = pre;
            p->ltag = 1;
        }
        if(pre!=NULL&&pre->rchild==NULL)
        {
            pre->rchild = p;     //建立前驱线索的后继线索
            pre->rtag = 1;
        }
        pre = p;       //标记当前结点成为刚刚访问过的结点
        InThread(p->rchild,pre);     //递归，线索化右子树
    }
}
```

线索二叉树的遍历

中序线索二叉树

主要为访问运算服务

这种遍历不需要借助栈，它结点中包含线索二叉树的前驱和后继信息

中序序列下的第一个结点

```
ThreadNode *Firstnode(ThreadNode *p)
{
    while(p->ltag == 0)
        p = p->lchild;     //最左下结点(不一定是叶结点)
        return p;
}
```

中序线索二叉树中结点p在中序序列下的后继结点

```
ThreadNode *Nextnode(ThreadNode *p)
{
    if(p->rtag == 0)
        return FirstNode(p->rchild);
    else
        return p->rchild;  //rtag == 1直接返回后继线索
}
```

不含头结点的中序线索二叉树的中序遍历算法

```
void InOrder(ThreadNode *T)
{
    for(ThreadNode *p = Firstnode(T);p!=NULL;P=Nextnode(p))
      visit(p);
}
```

配套课程公众号[考机研]
永久联系微信:4550060

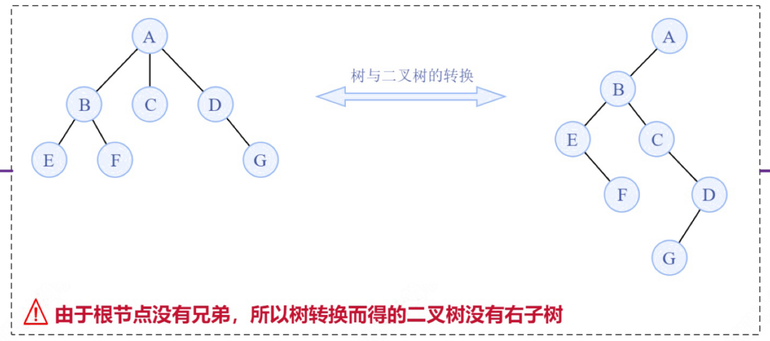
树和森林

树的存储结构

- 顺序存储
  - 双亲表示法
    - **增设尾指针，指示其双亲结点在数组中的位置。根结点下标为0，伪指针为-1**
    - 优点：快速查找双亲结点　O(1)
    - 缺点：求孩子时需要遍历整个结构
- 链式存储
  - 孩子表示法
    - **每个结点的孩子结点都用单链表链接起来形成一个线性结构**
    - **n个结点就有n个孩子链表（叶子结点的孩子链表为空表）**
    - 设置两种结点类型
      - 孩子链表的结点
      - 每个孩子链表的表头结点
    - 优点：易于查找孩子结点
    - 缺点：查找双亲麻烦，需要遍历n个结点中孩子链表指针所指向的n个孩子链表
  - 孩子兄弟表示法（二叉树表示法）
    - **设置两个指针**
      - 指向结点的第一个孩子结点
      - 指向该结点的下一个兄弟结点
    - **优点：易于查找孩子结点和兄弟，可以方便地实现树转换为二叉树地操作**
    - **缺点：查找双亲麻烦**

树和二叉树的转换

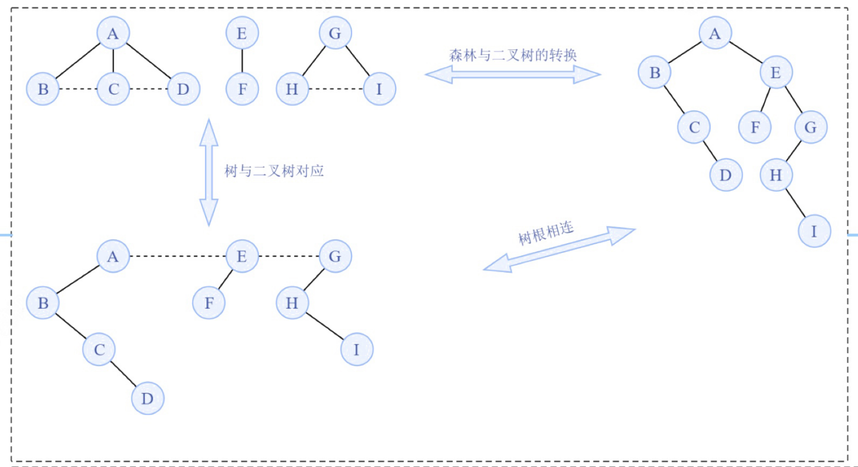
树→二叉树



⚠ **由于根节点没有兄弟，所以树转换而得的二叉树没有右子树**

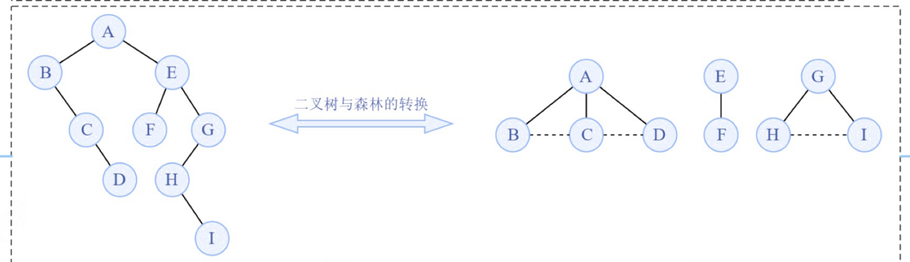
1. 每个结点左指针指向它的第一个孩子结点
2. 右指针指向它在树中的相邻兄弟结点

森林→二叉树



1. 将森林中的每棵树转换成相应的二叉树
2. 每棵树的根也可视为兄弟关系，在每棵树的根之间加一根连线
3. 以第一棵树的根为轴心顺时针旋转45°

二叉树→森林



二叉树

森林

1. 若二叉树非空，则二叉树的根及其左子树为第一棵树的二叉树形式，故将根的右链断开。
2. 二叉树根的右子树又可视为一一个由除第一棵树外的森林转换后的二叉树，应用同样的方法，直到最后只剩一棵没有右子树的二叉树为止
3. **最后再将每棵二叉树依次转换成树，就得到了原森林。**
   二叉树转换为树或森林是唯一 的

树和森林的遍历&并查集

树的遍历

- 先根遍历 —— 若树非空，先访问根结点，再从左到右的顺序遍历根结点的每棵子树
- 后根遍历 —— 若树非空，按从左到右的顺序遍历根节点的每棵子树，再访问根节点
- **树的先根遍历 == 二叉树先序遍历**
- **树的后根遍历 == 二叉树中序遍历**

森林的遍历

- 先序遍历 —— 若森林非空
  1. 访问森林中第一棵树的根结点
  2. 先序列遍历第一棵树中根结点的子树森林
  3. 先序遍历除去第一棵树之后剩余的树构成的森林
- 中序遍历 —— 若森林非空
  1. 中序列遍历第一棵树中根结点的子树森林
  2. 访问森林中第一棵树的根结点
  3. 先序遍历除去第一棵树之后剩余的树构成的森林
- **森林先序 == 二叉树先序**
- **森林中序 == 二叉树中序**

并查集

通常用树（森林）的双亲表示作为并查集的存储结构，树中的每个结点包含集合的一个元素，每棵树表示一个集合。多个集合形成一个森林，以每棵树的树根作为集合的代表，树中每个结点有一个指向双亲结点的指针，根结点的双亲结点指针指向其自身。

- 概念
  - 当给出两个元素的一个无序对（a,b）时，需要快速“合并”a和b分别所在的集合，这期间需要反复“查找”某元素所在的集合。
  - 在这种数据类型中，n个不同的元素被分为若干组，每组是一个集合，这种集合叫做分离集合，称之为**并查集**
- 基本操作
  1. Initials(S)：将集合S中的每个元素都初始化为只有一个单元素的子集合
  2. Union(S,Root1,Root2)：把集合S中的子集合Root2并入子集合Root1。要求Root1和Root2互不相交，否则不执行合并
  3. Find(S,x)：查找集合S中单元素x所在的子集合，并返回该子集合的根结点。

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

配套课程公众号[有机研]
永久联系微信:4550060

树与二叉树的应用（1）

二叉排序树（二叉查找树）

定义

- 空树（也是二叉排序树）
- 若非空，左子树结点值 < 根结点值 < 右子树结点值

二叉排序树的操作

查找

1. 从根节点开始，若二叉树非空，则将给定值 key 与根节点的关键字 T→data 查找，若相等，查找成功
2. 若不相等，则当根结点的关键字值 key 大于给的关键字 T→data 值时，在根节点的左子树中查找
3. 否则在根结点的右子树中查找

```
BSTNode *BST_Search(BiTree T,ElemType key,BSTNode *&p)
{
   //查找函数返回指向关键字值为key的结点指针，
   //若不存在，返回NULL
   p = NULL;   //p指向被查找结点的双亲，用于插入和删除操作中
   while(T!=NULL && key!=T->data)
   {
       p = T;
       if(key<T->data)
           T = T->lchild;
       else
           T = T->rchild;
   }
   return T;
}
```

二叉排序树的非递归查找算法

插入

1. 若原二叉树为空，则直接插入结点
2. 否则，若关键字 key 小于根结点 T→key，则插入左子树
3. 若关键字 k 大于根结点 T→key，则插入右子树

```
int BST_Insert(BiTree &T,KeyType k)
{  //在二叉排序树T中插入一个关键字为k的结点
   if(T==NULL)   //原树为空，新插入的记录为根结点
   {
      T = (BiTree)malloc(sizeof(BSTNode));
      T->key = k;
      T->lchild = T->rchild = NULL;
      return 1;
   }
   else if(k==T->key)  //树中存在相同的关键字
      return 0;        //插入shibai
   else if(k<T->key)   //插入T的左子树
      return BST_Insert(T->lchild,k);
   else                //插入T的右子树
      return BST_Insert(T->rchild,k)
}
```

二叉排序树插入算法

构造

1. 将二叉排序树T初始化为空树
2. 读入关键字为 key 的结点
3. 如果读入的关键字 key 不是输入结束的标志，循环执行操作
   1. 将此结点插入二叉排序树T中
   2. 读入关键字为 key 的结点

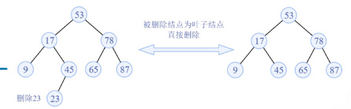
```
void Creat_BST(BiTree &T,KeyType str[],int n)
{
   //用关键字数字str[]建立一个二叉排序树
   T = NULL;
   int i = 0;
   while(i<n)   //依次将每个元素插入
   {
       BST_Insert(T,str[i]);
       i++;
   }
}
```

二叉排序树的构造算法

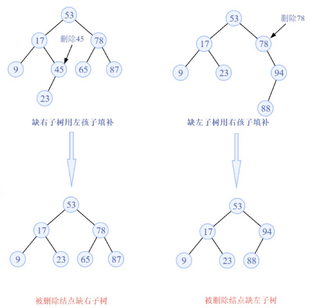
删除

三种情况下的删除过程

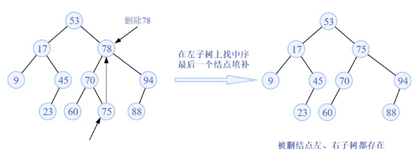
1. **若被删除结点z是叶结点，则直接删除，不会破坏二叉排序树的性质。**



2. **若结点z只有一棵左子树或右子树，则让z的子树成为z父结点的子树，替代z的位置。**



3. **若结点z有左、右两棵子树，则令z的直接后继(或直接前驱)替代z，然后从二叉排序树中删去这个直接后继(或直接前驱)，这样就转换成了第一或第 二种情况。**



查找效率分析

二叉排序树查找算法的平均长度主要取决于树的高度

- 只有左（右）孩子的单支树为O(n)
- 左右子树高度差绝对值不超过1为O($\log_2 n$)

二叉排序树与二分查找

区别和联系	**查找过程**	**平均性能分析**	**维护表的有序性**	**有序表**
二叉排序树	一样	查找不唯一	无需移动结点，只需修改指针即可插入和删除	动态查找
二分查找	一样	查找唯一	插入删除O(n)	静态查找

配套课程公众号[有机研]
永久联系微信:4550060

树与二叉树的应用（2）

平衡二叉树

定义

- 规定任意结点的左右子树高度差的绝对值不超过1
- 空树
- 左子树、右子树都是平衡二叉树，且左子树和右子树的高度差的绝对值不超过1
- 平衡因子只能是-1，0，1

操作

插入

1. LL平衡旋转（右单旋转）

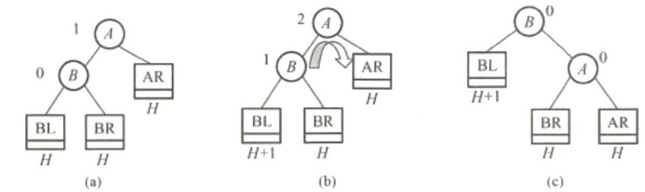
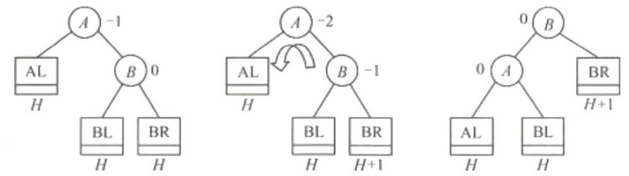
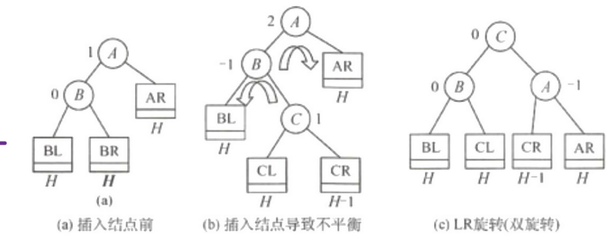


图 5.28　LL 平衡旋转

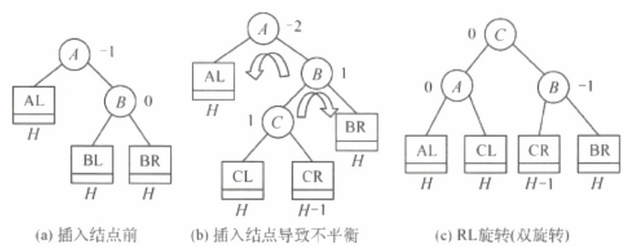
2. RR平衡旋转（左单旋转）



3. LR平衡旋转（先左后右双旋转）



4. RL平衡旋转（先右后左双旋）

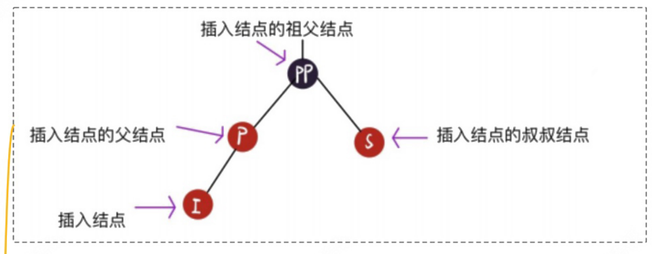


查找

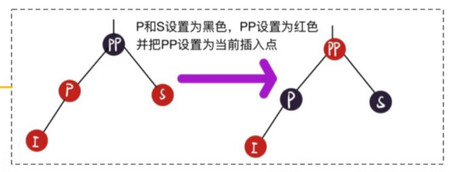
- 在平衡二叉树的查找过程与二叉排序树的相同
- 假设以Nh表示深度为h的平衡二叉树中含有的最少结点数，显然有
- 有 $n_0=0$，$n_1=1$，$n_2=2$，并且有 nh = n(h-1) + n(h-2) + 1 ——⚠ 注意：h为下标
- 平均查找长度为$O(\log_2 n)$

配套课程公众号[有机研]
永久联系微信:4550060

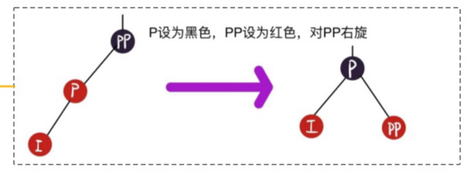
- 树与二叉树的运用
  - 红黑树RBT
    - 定义
      - 是一种含有红黑结点的自平衡二叉查找树 —— 左子树节点值≤根节点值≤右子树节点值
      - 与普通BST相比，它满足
        1. 每个结点或是红色，或是黑色的
        2. 根结点是黑色的
        3. 叶结点（外部结点、NULL结点）均是黑色的
        4. 每个红色结点的两个子结点一定都是黑色的
        5. 任意一结点到每个叶子结点的路径都包含数量相同的黑结点 —— 如果一个结点存在黑子结点，那么该结点肯定有两个子结点
      - 从某结点出发（不包含该结点）的任一简单路径上的黑色结点总数称为该结点的“黑高”（通常记为 bh），根结点的 bh 就为这棵红黑树的 bh
    - 性质（结论）
      - 根结点的最长路径不大于最短路径的两倍
      - 有 n 个内部结点的红黑树的高度 $h \le 2log_2(n+1)$ —— 红黑树的查找操作时间复杂度=$O(log_2 n)$
      - 适用于频繁的插入和删除操作



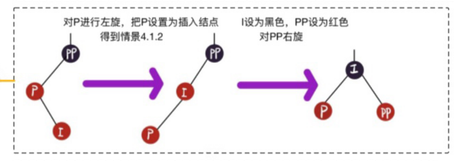
    - 查找 —— 与二叉排序树的查找无异
    - 插入
      1. 红黑树为空树 —— 把插入结点作为根结点，并把结点设置为黑色
      2. 插入结点的 Key 已存在 —— 把 插入结点设置为将要替代结点的颜色，再把结点的值更新就完成插入
      3. 插入结点的父结点为黑结点 —— 直接插入
      4. 插入结点的父结点为红结点
         - 叔叔结点存在并且为红结点



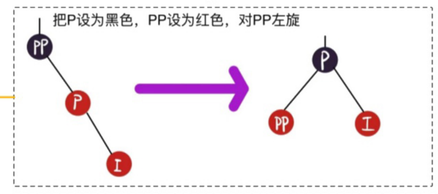
         - 叔叔结点不存在或为黑结点，并且插入结点的父亲结点是祖父结点的左子结点
           - 插入结点是其父结点的左子结点



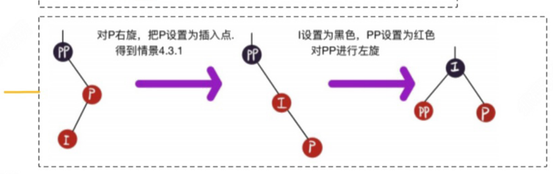
           - 插入结点是其父结点的右子结点



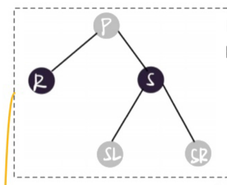
         - 叔叔结点不存在或为黑结点，并且插入结点的父亲结点是祖父结点的右子结点
           - 插入结点是其父结点的右子结点



           - 插入结点是其父结点的左子结点

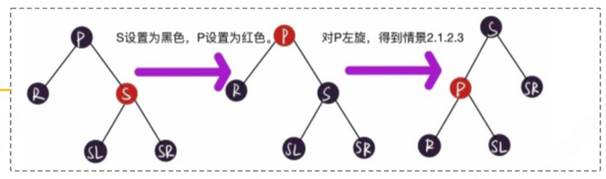


    - 基本操作
      - 删除

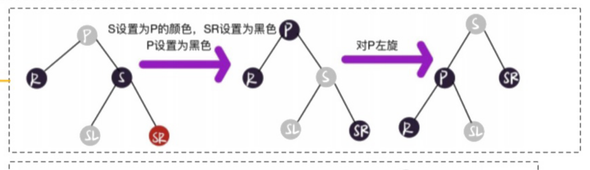


R表示替代结点，P表示替代结点的父结点，S表示替代结点的兄弟结点，SL表示兄弟结点的左子结点，SR 表示兄弟结点的右子结点。灰色结点表示它可以是红色也可以是黑色

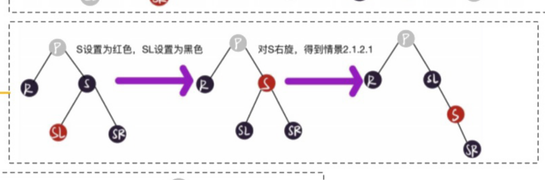
        - 替换结点
          - 若删除结点无子结点，直接删除
          - 若删除结点只有一个子结点，用子结点替换删除结点
          - 若删除结点有两个子结点，用后继结点（大于删除结点的最小结点）替换删除结点
        - 调整
          - 替换结点是红色结点
          - 替换结点是黑色结点
            - 替换结点是其父结点的左子结点
              - 替换结点的兄弟结点是红结点



              - 替换结点的兄弟结点是黑结点
                - 替换结点的兄弟结点的右子结点是红结点，左子结点任意颜色

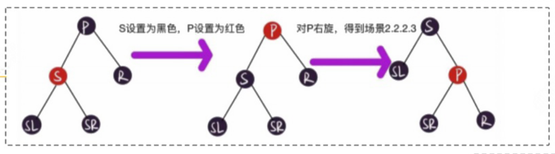


                - 替换结点的兄弟结点的右子结点为黑结点，左子结点为红结点

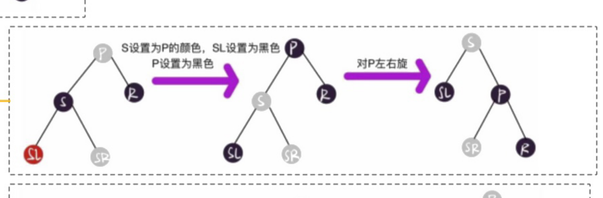


                - 替换结点的兄弟结点的子结点都为黑结点

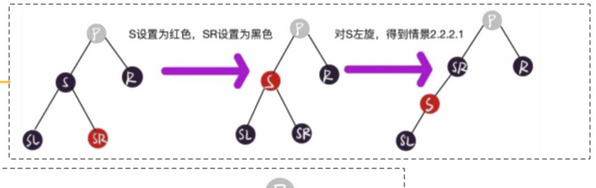
            - 替换结点是其父结点的右子结点
              - 替换结点的兄弟结点是红结点



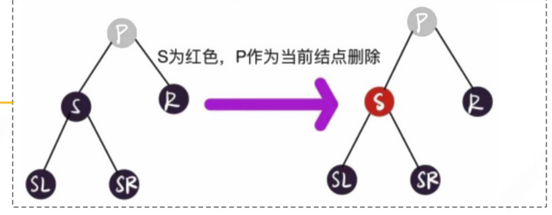
              - 替换结点的兄弟结点是黑结点
                - 替换结点的兄弟结点的左子结点是红结点，右子结点任意颜色



                - 替换结点的兄弟结点的左子结点为黑结点，右子结点为红结点



                - 替换结点的兄弟结点的子结点都为黑结点



配套课程公众号[有机研]
永久联系微信:4550060

线上5分/页打印，包邮到家
操作简单，一看就会

树与二叉树的应用（3）

哈夫曼树和哈夫曼编码

- 哈夫曼树的定义
  - 权 —— 树结点被赋予某种意义的数值
  - 结点的带权路径长度 —— 从根结点到该结点的路径长度*该结点的权值
  - 树的带权路径长度 —— 所有叶子结点的带权路径长度之和WPL
  - **哈夫曼树/最优二叉树** —— WPL最小的二叉树
- 哈夫曼树的构造
  1. **将这n个结点分别作为n棵仅含有一个结点的二叉树，构成森林F**
  2. **构造一个新结点，从F中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树根结点的权值之和**
  3. **将F中删除刚刚选出的两棵树，同时将新得到的树加入F中**
  4. **重复上述的2，3步骤，直至F中只剩下一棵树为止**
- 哈夫曼树的特点
  - 每个初试结点最终都成为叶结点，且权值越小的结点到根节点的路径长度越大
  - **构造过程中共新建了 n-1 个结点，因此哈夫曼树的结点总数为2n-1**
- 哈夫曼编码
  - 固定长度编码 —— 每个字符都用同样位数的二进制表示
  - 可变长度编码 —— 不同位数二进制表示
  - **前缀编码**
    - **没有一个编码是另外一个编码的前缀**
    - **哈夫曼编码就是前缀码**
  - **构建过程**
    1. **将每个字符作为一个独立的结点，权值为它出现的频度/次数，然后构建哈夫曼树**
    2. **所有结点都出现在叶子结点上，用0标记左孩子，1标记右孩子**
    3. **按照从根结点走向叶子结点的路径上的1，0来组成编码**

配套课程公众号[有机研]
永久联系微信:4550060

图的定义

- 图的定义
  - 图由顶点集V和边集E组成，记为G（V,E）
  - G（V,E） 其中V为顶点集，E为边集
- 无向图
  - 若E是无向边的有限集合时，则图G为无向图
- 有向图
  - 若E是有向边（也称为弧）的有限集合时，则图G为有向图
- 简单图
  - 1 不存在重复的边
  - 2 不存在顶点到自身的边
- 相对
- 多重图
  - 若图G中某两个结点之间的边数多于一条，又允许顶点通过同一条边和自己关联
- 完全图
  - 有向完全图 —— 任意两个顶点之间存在方向相反的两条弧 —— **n*(n-1)条弧边**
  - 无向完全图 —— 任意两个顶点之间都存在边 —— **n*(n-1)/2条边**
- 子图
  - **设有两个图G（V,E）和G'=(V',E'),若V'是V的子集，且E'是E的子集则称G'是G的子图**
- 连通图
  - 连通 —— 若从顶点V到顶点W有路径存在，则称V和W是连通的
  - 连通图 —— 任意两个顶点都是联通的
  - 非连通图 —— 如果一个图中有n个顶点，并且有小于n-1条边，则此图必是非连通图
  - **极大连通图（连通分量）**
    - 1 是一个子图
    - 2 是连通的
    - 3 顶点足够多
    - 4 极大连通子图包含依附这些顶点的所有边
  - 强连通 —— 从顶点V到W，和从顶点W到V之间都有路径
- ★ **生成树**
  - 1 **连通图的生成树是包含图中的全部顶点的一个极小连通子图**
  - 2 **若图中顶点数为n，则它的生成树含有n-1条边**
  - 3 **生成树去掉一条边则变成非连通图，加上一条边就会有回路**
- 度
  - 无向图 —— 度之和 = 2倍的边数
  - 有向图
    - 入度
    - 出度
    - 入读和 = 出度和 = 边数
    - 度之和 = 出度+入度
- 其他概念
  - 权和网 —— 每一条边都可以标上具有某种含义的数值，该数值称为该边的权值
  - 稠密图、稀疏图 —— 边数很少的称为稀疏图，反之称为稠密图
  - 路径，路径长度和回路
    - **路径**：顶点到另外一个顶点的序列
    - **路径长度**：路径上边的数目
    - **回路（或环）**：第一个顶点和最后一个顶点相同的路径
  - 简单路径，简单回路
    - 简单路径 —— 顶点不重复
    - 简单回路 —— 除第一个顶点和最好一个顶点外，其余顶点不重复出现的回路
  - 距离 —— 从顶点u到顶点v的最短路径存在，则此路径长度为距离；没有则为∞
  - 有向树 —— 一个顶点的入度为0，其余顶点的入度均为1的有向图

配套课程公众号[有机研]
永久联系微信:4550060

- 图
  - 图的存储与基本操作
    - 顺序存储结构——邻接矩阵法
      - 定义
        - 一维数组存储顶点信息
        - 二维数组存储边或弧信息（即各顶点之间的邻接关系）
      - A[i][j] =
        - **1，顶点到另一个的边或弧**
        - **0，顶点与这个顶点没有边或弧**
      - 邻接矩阵的作用
        - 无向图
          1. 判断两个顶点是否有边
          2. 找到某个顶点的邻接点
          3. **求某个顶点的度 = 该顶点行/列为1的个数**
          4. **无向图的邻接矩阵是对称矩阵** — 可以压缩存储
        - 有向图
          1. 判断某个顶点是否有弧
          2. 找到某个顶点的临界点
          3. **该顶点的入度 = 该列的1的个数**
          4. **该顶点的出度 = 该行的1的个数**
        - 注意
          - 邻接矩阵表示法的空间复杂度为O（n^2），其中有n个顶点
          - **稠密图适用于用邻接矩阵存储**
          - 缺点：要确定有多少条边需要按行按列对每个元素进行检测
    - 链式存储结构——邻接表
      - 顶点表 — **data(顶点域) | firstarc(边表头指针)**
      - 边表 — **adjvex(邻接点域) | nextarc(指针域)**
      - 无向图
        - **度之和 = 边数 = 边表中所有顶点和的1/2**
        - **某个顶点的度 = 该顶点边表里的顶点数**
      - 有向图
        - **某个顶点的出度 = 该链出边表的顶点个数**
        - **出度之和 = 出边表中所有顶点之和**
    - 十字链表
      - **十字链表有向图的一种链式存储结构**
      - 在十字链表中容易求得顶点的出度和入度
      - 图的十字链表的表示不唯一，但一个十字链表表示确定一个图
    - 邻接多重表
      - **邻接多重表是无向图的另外一种链式存储结构**
      - 在邻接表中，容易求得顶点和边的各种信息
      - 注意：无向图的邻接多重表表示法，每条边只有一个顶点
      - 边表顶点个数 = 边数

配套课程公众号[有机研]
永久联系微信:4550060

图的遍历

广度优先遍历

广度优先遍历类似于树的按层次遍历的过程

算法步骤

- **从图中某个顶点 v 出发， 访问 v, 并置 visited[v]的值为 true, 然后将 v 进队**
- **只要队列不空， 则循环下述操作**
  - 队头顶点v出队
  - 依次检查 v 的所有邻接点 w，
    如果 visited[w]的值为 false,
    则访问 w, 并置 visited[w]的值为 true,
    然后将 w 进队

```
void BFS(Graph G, int v)
{
    //从顶点 v 出发,广度优先遍历图G,算法借助一个辅助队列Q
    visit(v);    //访问初始顶点
    visited[v] = TRUE;     //对v做已访问标记
    EnQueue(Q,v);       //顶点v入队
    while(!QueueEmpty(Q))
    {
        DeQueue(Q,v);    //顶点v出队列
        for(w = FirstNeighbor(G,v); w>=0; w = NextNeighbor(G,v,v))
            //检测v的所有邻接点
            if(!visited[w])    //w为v的尚未访问的邻接顶点
            {
                visit(w);      //访问顶点w
                visited[w] = TRUE;  //对w做已访问顶点标记
                EnQueue(Q,w);    //顶点w做入队处理
            }//if
    }//while
}
```

广度优先遍历非递归算法

BFS算法性能分析

1. 时间复杂度
   - **邻接表：O(|V|+|E|)**
   - **邻接矩阵：O(|V²|)**
2. 空间复杂度
   - 邻接表
   - 邻接矩阵

   → **O(|V|)**

BFS算法可以求解单元最短路径问题

广度优先生成树

- 邻接表存储 —— 表示不唯一
- 邻接矩阵存储 —— 表示唯一

深度优先遍历

深度优先遍历类似于树的先序遍历，是树的先序遍历的推广

算法步骤

- 从图中某个顶点 v 出发，访问 v, 并置 visited[v]的值为 true
- 依次检查 v 的所有邻接点 w, 如果 visited[w]的值为 false,
  再从 w 出发进行递归遍历，直到图中所有顶点都被访问过
- 注意：若是非连通图， 上述遍历过程执行之后， 图中一定还有顶点未被访问，
  需要从图中另选一个未被访问的顶点作为起始点， 重复上述深度优先搜索过程，
  直到图中所有顶点均被访问过为止

```
bool visited[MAX_VERTEX_NUM];  //标记访问数组
void DFSTraverse(Graph G)
{
    //对图G进行深度优先遍历，访问函数为visite()
    for(v = 0; v<G.vexnum; v++)
        visited[v] = FALSE;     //对v做已访问标记
    for(v = 0; v<G.vexum; v++)
        if(!visited[v])
            DFS(G,v);
}

void DFS(Graph G, int v)
{
    //从顶点v出发，采用递归思想，深度优先遍历图G
    visit(v);
    visited[v] = TRUE;
    for(w = FirstNeighbor(G,v); w>=0; w = NextNeighbor(G,v,v))
        if(!visited[w])    //w为v的尚未访问的邻接顶点
            DFS(G,w)
}
```

深度优先遍历递归算法

DFS算法性能分析

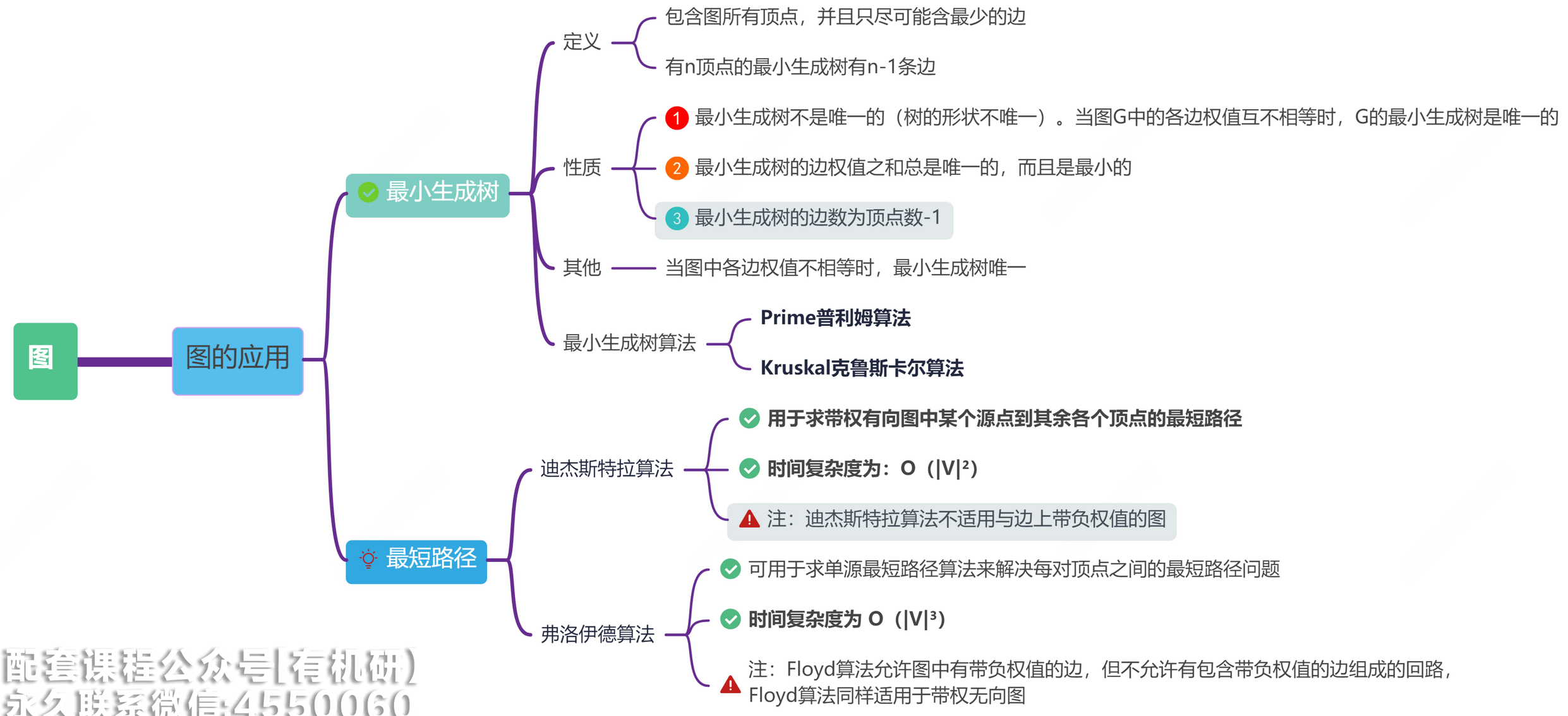
1. 时间复杂度
   - **邻接表：O(|V|+|E|)**
   - **邻接矩阵：O(|V|²)**
2. 空间复杂度
   - 邻接表
   - 邻接矩阵

   → **O(|V|)**

深度优先生成树和生成森林

- 连通图 —— 生成深度优先树
- 非连通图 —— 生成深度优先森林
- 邻接表存储 —— 表示不唯一
- 邻接矩阵存储 —— 表示唯一

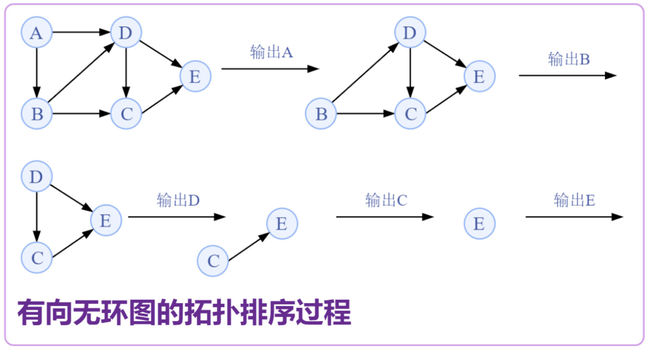
配套课程公众号[有机研]
永久联系微信:4550060



配套课程公众号[有机研]
永久联系微信:4550060

拓扑排序

- 有向无环图（DAG）
  - 一个有向图中不存在环或回路，称为有向无环图
- AOV网（Activity On Vertex）
  - 活动在顶点上
  - 在AOV网中，不应该出现有向环
  - 一个活动完成后才能进行下一个活动，具有传递性；任何活动不能以它自己作为自己的前驱或后继
- 拓扑排序的定义
  - 每个顶点只出现一次
  - 若顶点A在序列中排在顶点B的前面，则在图中不存在从顶点A到B的路径
  - **注：每个DAG图有一个或多个拓扑排序序列**
- 拓扑排序的过程及思想
  1. **从AOV图中选择一个入度为0（即没有前驱）的顶点并输出**
  2. **然后删除此顶点，并删去以此为起点的有向边（删除出度）**
  3. **重复以上①②步骤，直到所有的顶点都输出或者找不到入度为0（无前驱）的点**



有向无环图的拓扑排序过程

- 逆拓扑排序的过程及思想
  1. 从AOV 网中选择一个没有后继（出度为0）的顶点并输出
  2. 从网中删除该顶点和所有以它为终点的有向边
  3. 重复①和②直到当前的 AOV 网为空
- 时间复杂度 —— O(|V|+|E|)

配套课程公众号[有机研]
永久联系微信:4330060

关键路径

- AOE（活动在边上）
  - 以顶点表示事件，以有向边表示活动
  - 边上的权值表示完成活动所需的时间
- 概念
  - 开始顶点（源点） —— 仅有一个入度为0的顶点 —— 工程开始
  - 结束顶点（汇点） —— 仅有一个出度为0的顶点 —— 工程结束
  - **关键路径** —— **从源点到汇点的所有路径中，具有最大路径长度的路径**
  - **关键活动** —— **关键路径上的活动**
  - **关键路径的长度** —— **完成整个工程的最短时间**
- 参量的定义
  1. 时间V(k)的最早发生时间Ve(k)
  2. 事件V(k)的最迟发生时间Vl(k)
  3. 活动ai的最早发生时间e(i)
  4. 活动ai的最迟发生时间l(i)
  5. 一个活动ai的最迟开始事件l(i)和其最早开始时间e(i)的差额
- 求关键路径的过程
  1. 对图中顶点进行拓扑排序，在排序过程中按拓扑排序求出每个事件的**最早发生时间Ve(i)**
  2. 按逆拓扑排序列求AOE网络中每个事件的**最迟发生时间vl(i)**
  3. 求AOE网中每个活动的**最早开始时间e(i)**
  4. 求AOE网中每个活动的**最迟开始时间l(i)**
  5. 找出 e(i)= l(i)的活动ai，即为关键活动。由关键活动形成的由源点到汇点的每一条路径就是关键路径， 关键路径有可能不止一条 。
- 注意点
  - **关键路径上所有的活动都是关键活动，它是决定整个工程的关键因素，因此可通过加快关键活动来缩短整个工程的工期。但也不能任意缩短关键活动，因为一旦缩短到一定的程度，该关键活动就可能变成非关键活动。**
  - **关键路径并不唯一，且对于有几条关键路径的网，只提高一条关键路径上的关键活动并不能缩短整个工期，只有加快那些包括在所有关键路径上的关键活动才能达到缩短工期的目的**

配套课程公众号[有机研]
永久联系微信:4550060

查找

查找基本概念

- **静态查找**
  - 查询某个数据元素是否在查找表中
  - 检索数据元素的各种属性
- **动态查找元素** —— 除了上述两种操作外还需要进行修改和删除数据元素的操作
- 关键字 —— 数据元素中唯一标识该元素的某个数据项的值
- 平均查找长度（ASL） —— 所有查找过程中进行关键字比较次数的平均值

顺序查找

- 无序线性表 —— 从线性表的一端，逐个检查关键字是否满足给定条件
- 平均时间只与元素存储位置有关
- **平均查找长度ASL —— （n+1）/2**
- **缺点 —— 当n比较大时，平均查找长度较大，效率低**

```c
typedef struct{
    ElemType *elem;      //元素存储空间基址
    int TableLen;        //当前长度
}SSTable;
int Seach_Seq(SSTable ST,KeyType key)
{
    ST.R[0].key = key;      //哨兵
    for(i = ST.length; ST.R[i].key!=key; i--)  //从后往前找
    return i;    //若表中不存在关键字为key的元素，将查找到i为0时退出循环
}
```

顺序查找算法

折半查找（二分查找）

- **查找基本思想及过程**
  - 首先将给定值key与表中位置元素的关键之比较
  - 若相等，则查找成功
  - 若不相等
    - key>中间位置 —— 去后半部分，low = mid+1
    - key<中间位置 —— 去前半部分，high = mid-1
- **算法步骤**
  1. **置查找区间初值，low = 1,high = length(表长)**
  2. **当 low ≤ high 时，循环执行以下操作：**
     - **mid取值为low和high的中间值**
     - **将给定值key与中间位置记录的关键字进行比较，若相等则查找成功，返回中间位置皿d**
     - **若不相等则利用中间位置记录将表对分成前、后两个子表 。如果key比中间位置记录的关键字小，则 high取为mid-1 否则 low取为mid+l**
  3. **循环结束，说明查找区间为空，则查找失败，返回0**

```c
int Seach_Bin(SSTable ST,KeyType key)
{
    low = 1;high = ST.length;          //置初始区间初值
    while(low <= high)
    {
        mid = (low + high)/2;          //取中间位置
        if(key == ST.R[mid].key)
            return mid;                //查找成功则返回所在位置
        else if(key < ST.R[mid].key)
            high = mid-1;              //从前半部分继续查找
        else
            low = mid+1;               //从前半部分继续查找
    }
    return 0;                          //查找失败，返回0
}
```

折半查找算法

- **构造判定树**
  1. 表顺序表中的中间位置作为根结点
  2. 左子树根结点 == 左半边的中间元素
  3. 右子树根结点 == 右半边的中间元素
- 平均查找长度
  - 查找成功
    - 比较次数 —— 结点在判定树的层次
    - 结点
  - 查找失败
    - 比较位置 —— 叶子结点
    - 比较次数 —— 结点所在判定树的层次

分块查找（索引顺序查找）

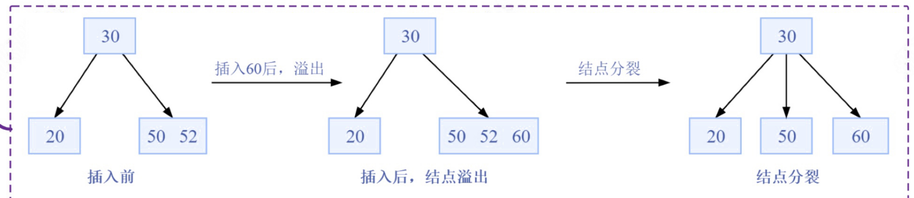
- 基本思想 —— 将查找表分为若干子块，块内元素可以无序，但块间是有序的
- 查找过程
  1. 在索引表中确定待查记录所在的块，可以顺序查找或折半查找
  2. 块内顺序查找
- **平均查找长度**
  - 性能介于顺序查找和折半查找之间的一种查找方法
  - 采取顺序查找 —— **ASL = （s^2 + 2s + n）/2**
  - 索引表折半查找 —— **ASL = ⌈$\log_2$（b+1）⌉ + （s+1）/2**
- 分块查找的优缺点
  - **优点**
    - 在表中插入和删除数据元素时，只要找到该元素对应的块，就可以在该块内进行插入和删除运算
    - 由于块内是无序的，故插入和删除比较容易，无需进行大量移动
  - **缺点** —— 要增加一个索引表的存储空间并对初始索引表进行排序运算

配套课程公众号[有机研]
永久联系微信:4550060

B树和B+树

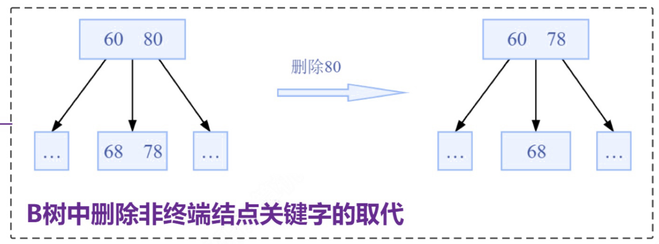
B树（多路平衡查找树）

- 定义 — B树的阶 — B树中指针数量最大的
- 一棵m阶B树，或为空树，或满足下列条件
  - 树的每个结点至多m棵子树（**即至多含有m-1个关键字**）
  - 若根结点不是终端结点（叶子结点），则至少有两棵子树；（**至少一个关键字**）
  - 除根结点外的所有非终端结点至少有「m/2」（**即至少含有「m/2」-1个关键字**）
- 基本操作
  1. B树的查找
     - B树的查找和二叉树排序很类似
     - 查找过程
       1. 先让等待查找关键字Key和结点的关键字比较，如果等于，则查找成功
       3. 若不等于任何一个关键字，则看key处在哪个范围内，然后去对应的指针所指向的子树中查找
       3. 如果key比第一个关键字还小，则去它的左边指针所指的子树查找，如果比最后一个关键字还大，则去它右边指针所指向的子树中查找
  2. **B树的插入** — 分裂的方法
     - **取这个关键字数组中的中间关键字「n/2」作为一个新的结点，然后其他关键字形成两个结点作为新结点的左右孩子**



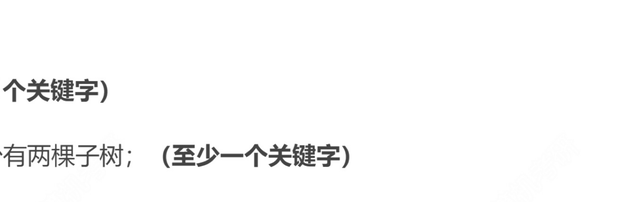
结点分裂示意图

  3. B树的删除
     - **被删除关键字 k 不在终端结点上时**

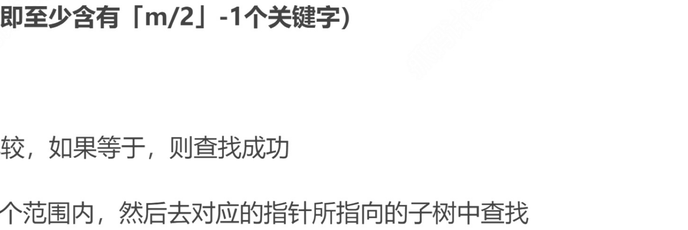


B树中删除非终端结点关键字的取代

     - **被删除的关键字 k 在终端结点上时**
       1. 直接删除关键字
       2. 兄弟够借



       3. 兄弟不够借



B+树

- B树与B+树的区别（重点）
  1. 关键字
     - B+树中，具有n个关键字的结点只含有n棵子树，即每个关键字对应一棵子树
     - B树中，具有n个关键字的结点含n+1棵子树
  2. 结点范围
     - B+树中，每个结点（非根结点）关键字个数 n 的范围 ⌈m/2⌉ <= n <= m
     - B树中，非根结点关键字的个数n的范围 ⌈m/2⌉-1<= n <= m-1
  3. 叶子结点
     - B+树中，叶结点包含信息，所有非叶结点仅起索引作用，对应存储子树中最大关键字
     - B树中，叶子结点不包含信息
  4. 关键字
     - B+树，叶结点包含了全部关键字，即在非叶结点中出现的关键字也会出现在叶结点中
     - B树中，叶结点包含的关键字和其他结点包含的关键字是不可以重复的
  5. 在B+树中有一个指针指向关键字最小的结点，所有叶子结点连接成一个单链表

配套课程公众号[有机研]
永久联系微信:4550060

散列表（Hash）

散列表的基本概念

- 散列表
  - 根据关键字而直接进行访问的数据结构
  - **散列表建立了关键字和存储地址之间的一种直接映射关系**
- 散列函数 —— 一个把查找表中的关键字映射成该关键字对应的地址的函数，记为 Hash(key) = Addr
- 冲突 —— 对不同关键字可能得到同一散列地址，即 $key_1 \neq key_2$，而 $H(key_1) = H(key_2)$，这种现象称为冲突
- 同义词 —— 具有相同函数值的关键字对该散列函数来说称作同义词，$key_1 \neq key_2$互称为同义词

散列函数的构造

- **构造函数的注意事项**
  1. 定义域：全部需要存储的关键字；值域：依赖于散列表的大小或地址范围
  2. **散列函数计算出来的地址应尽可能等概率，均匀分布，尽可能减少冲突**
  3. **函数计算简单，能够计算出任一关键字对应的散列函数**

  → **常考好的散列函数的构造原则**
- 1 **直接地址法**
  - 直接去关键字的某个线性函数值为散列地址，直线公式 H(key) = a×key + b （a和b是常数）
  - 这种方法计算最简单，且不会产生冲突 它适合关键字的分布基本连续的情况，若关键字分布不连续，空位较多，则会造成存储空间的浪费。
- 2 **除留余数法**
  - 散列表长为m，取一个不大于m，但是最接近于或等于m的质数p
  - 散列函数为：H(key) = key%p
  - **除留余数法的关键是选好p，使得每个关键字通过该函数转换后等概率地映射到散列空间上地任一地址，从而减小冲突**
- 3 数字分析法
  - 如果事先知道关键字集合， 且每个关键字的位数比散列表的地址码位数多，每个关键字由n位数组成，如k1...kn，则可以从关键字中提取数字分布比较均匀的若干位作为散列地址
  - **数字分析法的适用情况：**事先必须明确知道所有的关键字每一位上各种数字的分布情况
- 4 平方取中法
  - 取关键字的平方值的中间位作为散列地址
  - **平方取中法的适用情况：**不能事先了解关键字的所有情况，或难于直接从关键字中找到取值较分散的几位
- 5 折叠法 —— 将关键字分割成位数相同的几部分（最后一部分的位数可以不同），然后取这几部分的叠加和（舍去进位） 作为散列地址，这种方法称为折叠法

处理冲突碰撞的方法

- **开放地址法**
  Hi = (H(key)+di)%m
  - 1 **线性探测法**
    - 特点：冲突发生时，顺序查看表中下一单元，直到找出一个空闲单元或查遍全表
    - ⚠ **缺点：大量元素在相邻地址处堆积“聚集”，降低查找效率**
  - 2 **平方探测法** —— ⚠ **缺点：不饿能探测到所有单元造成空闲，但至少能探测到一般单元**
  - 3 再散列法
    - 设置两个散列函数
    - Hi = (H(key)+i*Hash2(key))%m
    - i是冲突的次数，初始为0；m为表长
  - 4 伪随机序列方法
- **拉链法（链地址法）**
  - 把具有相同散列地址的记录放在同一个单链表中，称为同义词链表
  - **拉链法适用于经常进行插入和删除的情况**

散列查找及性能分析

- 散列表查找
  - 与构造散列表过程基本一致
  - 先根据散列函数计算出散列地址，然后看看这个位置有没有关键字
  - 如果没有，查找失败
  - 如果有，则检查该记录是否等于关键字
    1. 等于关键字，查找成功
    2. 不等于，按照给定的冲突办法计算下一个散列地址，再用该地址重复上述过程
- **性能分析**
  - 装填因子α —— 定义为一个表的装满程度
    - **α = n/m**
    - **n:表中记录数**
    - **m:散列表长度**
    - **α越大，说明表越满**
  - 查找效率 —— 取决于散列函数，处理冲突的方法，填装因子α
  - 平均查找长度 —— 取决与装填因子α，而不直接依赖与 n（表中记录数） 或 m（散列表长度）

配套课程公众号[有机研]
永久联系微信:4550060

插入排序

直接插入排序

- 排序思想：首先以第一个元素为有序序列，将后面的元素一次插入到有序的序列中合适的为止，直到都插入有序序列
- 代码实现

```
void InsertSort(ElemType A[],int n)
{
    int i,j;
    for(i = 2; i<=n; i++)           //依次将A[2]~A[n]插入到前面已排序序列
        if(A[i] < A[i-1])           //若A[i]关键码小于其前驱，将 A[i]插入有序表
        {
            A[0] = A[i];            //复制为哨兵，A[0]不存放元素
            for(j = i-1; A[0]<A[j]; --j) //从后往前查找待插入位置
                A[j+1] = A[j];      //向后挪位
            A[j+1] = A[0];          //复制到插入位置
        }
}
```

- 性能分析
  1. 空间复杂度：仅用常熟个辅助单元，A[0]为哨兵，**故空间复杂度为O(1)**
  2. **时间复杂度**
     - **最好O(n)**：表中元素已经有序，此时每插入一个元素，都只需要比较一次而不需要移动元素
     - **最坏O(n²)**：表中元素顺序刚好与排序结果中的元素顺序相反，总的比较次数达到最大，总的移动次数也达到最大
     - **平均 O(n²)**
  3. 稳定性：**稳定**
  4. 适合性：**顺序存储，链式存储的线性表**

     注：大部分排序算法仅适用于顺序存储的线性表

折半插入排序

- 排序思想：将比较和移动操作分离，先找出插入位置，然后统一移动待插入位置的所有元素
- 代码实现

```
void InsertSort(ElemType A[], int n)
{
    int i,j,low,high,mid;
    for(i = 2; i<=n; i++)           //依次将A[2]~A[n]插入到前面的已排序序列
    {
        A[0] = A[i];                //将A[i]暂存到A[0]
        low = 1; high = i-1;        //设置查找范围
        while(low <= high)          //折半查找
        {
            mid = (low+high)/2; //取中间点
            if(A[mid] > A[0])
                high = mid-1;   //查找左半边子表
            else
                low = mid+1;    //查找右半子表
        }
        for(j = i-1; j>=high+1; --j)
            A[j+1] = A[j];          //统一后移动元素，空出插入位置
        A[high+1] = A[0];           //插入操作
    }
}
```

- 性能分析
  1. 时间复杂度：**O(n²)**
  2. 稳定性：**稳定**

希尔排序

- 排序思想：**将待排序表分成若干子表，对若干子表分别进行直接插入排序，当整张表呈“基本有序”时，再对全体记录进行一次直接插入排序**
- 代码实现（不需要掌握）
- 性能分析
  1. 时间复杂度：最坏情况下：**O(n²)**
  2. 空间复杂度：**O(1)**：仅使用了常数个辅助单元
  3. 稳定性：**不稳定**
  4. 适用性：仅适用于线性表为顺序存储的情况

配套课程公众号[有机研]
永久联系微信:4550060

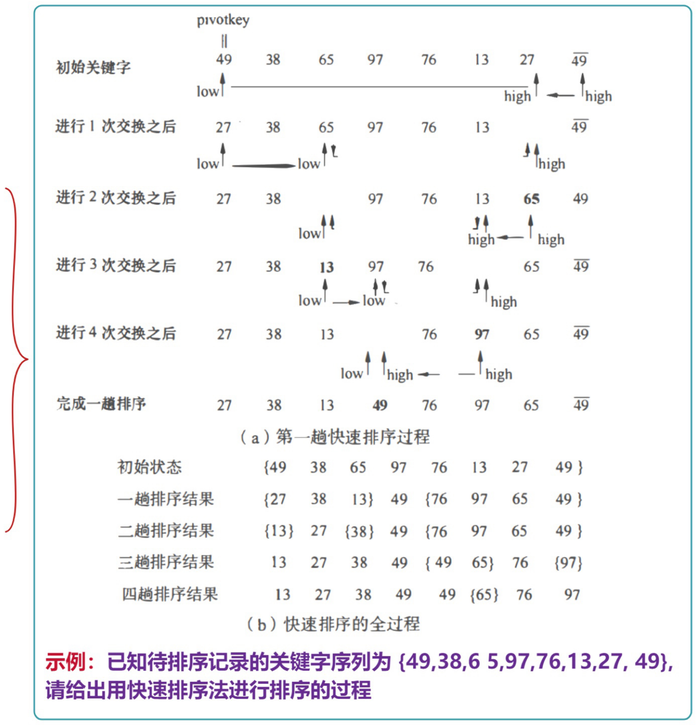
交换排序

冒泡排序

- 排序思想 —— 两两比较相邻元素的值，若为逆序则交换它们，直到序列比较完，此为第一躺冒泡；结果是将最小元素交换到待排序序列的第一个位置
- 代码实现 —— 多种方法（掌握并且手动实现）
- **性能分析**
  1. 时间复杂度
     - 最好情况下 —— O(n)
     - 最坏情况下 —— O(n²)
     - 平均情况 —— O(n²)
  2. 空间复杂度 —— O(1) —— 仅使用常数个辅助单元
  3. 稳定性 —— 稳定

快速排序

- 排序思想
  1. **在无序表中选取一个元素作为基准pivot，通常是第一个元素**
  2. **设置两个指针low,high分别指向无序表两端**
  3. high从右向左找比基准小的元素然后替换low所指向元素
  4. low从左向右找比基准大的元素然后替换high所指元素
  5. 重复步骤②和③，直到low和high相等为止。此时 low 或 high 的位置即为枢轴在此趟排序中的最终位置， 原表被分成两个子表



（a）第一趟快速排序过程

初始状态	{49	38	65	97	76	13	27	49 }
一趟排序结果	{27	38	13}	49	{76	97	65	49 }
二趟排序结果	{13}	27	{38}	49	{76	97	65	49 }
三趟排序结果	13	27	38	49	{ 49	65}	76	{97}
四趟排序结果	13	27	38	49	49	{65}	76	97

（b）快速排序的全过程

示例：已知待排序记录的关键字序列为 {49,38,6 5,97,76,13,27, 49}，请给出用快速排序法进行排序的过程

- 代码实现

```
void QuickSort(ElemType A[],int low,int high)
{
    if(low<high)
    {
        int pivotpos = Partition(A,low,high);
        QuickSort(A,low,pivotpos-1);
        QuickSort(A,pivotpos+1,high);
    }
 }
int Partition(ElemType A[],int low,int high)
{   //非递归算法
    //严版教材的划分算法（一趟划分过程）
    ElemType pivot = A[low]; //将当前表中第一个元素设置为枢纽值，对表进行划分
    while(low < high)
    {
        while(low<high&&A[high]>=pivot)
           high--;
        A[low] = A[high];   //将比枢轴值小的元素移动到左端
        while(low<high&&A[low]<=pivot)
           low++;
        A[high] = A[low];  //将比枢轴值大的元素移动到右端
     }
     A[low] = pivot;  //枢轴元素存放到最终位置
     return low;   //返回存放枢轴的最终位置
 }
```

重点掌握

- 性能分析
  1. 时间复杂度
     - 最坏情况下 —— O(n²)
     - 平均情况下 —— O(nlog₂n)
  2. 空间复杂度
     - 最好情况下 —— O(log₂n)
     - 最坏情况下 —— O(n)
     - 平均情况 —— O(log₂n)
  3. 稳定性 —— 不稳定

配套课程公众号[有机研]
永久联系微信:4550060

配套课程公众号[有机研]
永久联系微信:4550060

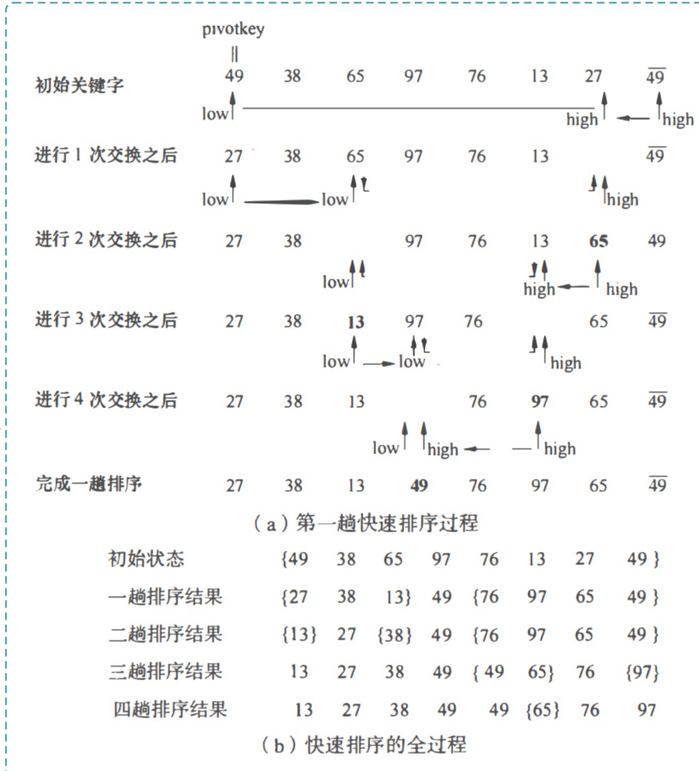
交换排序

冒泡排序

- 排序思想：两两比较相邻元素的值，若为逆序则交换它们，直到序列比较完，此为第一躺冒泡；结果是将最小元素交换到待排序序列的第一个位置
- 代码实现：多种方法（掌握并且手动实现）
- **性能分析**
  1. 时间复杂度
     - 最好情况下：O(n)
     - 最坏情况下：O(n²)
     - 平均情况：O(n²)
  2. 空间复杂度：O(1)——仅使用常数个辅助单元
  3. 稳定性：稳定

快速排序

- 排序思想
  1. **在无序表中选取一个元素作为基准pivot，通常是第一个元素**
  2. **设置两个指针low,high分别指向无序表两端**
  3. high从右向左找比基准小的元素然后替换low所指向元素
  4. low从左向右找比基准大的元素然后替换high所指元素
  5. 重复步骤②和③，直到low和high相等为止。此时 low 或 high 的位置即为枢轴在此趟排序中的最终位置， 原表被分成两个子表



（a）第一趟快速排序过程

初始状态	{49	38	65	97	76	13	27	49 }
一趟排序结果	{27	38	13}	49	{76	97	65	49 }
二趟排序结果	{13}	27	{38}	49	{76	97	65	49 }
三趟排序结果	13	27	38	49	{ 49	65}	76	{97}
四趟排序结果	13	27	38	49	49	{65}	76	97

（b）快速排序的全过程

示例：已知待排序记录的关键字序列为 {49,38,6 5,97,76,13,27, 49}，请给出用快速排序法进行排序的过程

- **代码实现**

```
void QuickSort(ElemType A[],int low,int high)
{
    if(low<high)
    {
        int pivotpos = Partition(A,low,high);
        QuickSort(A,low,pivotpos-1);
        QuickSort(A,pivotpos+1,high);
    }
 }
int Partition(ElemType A[],int low,int high)
{   //非递归算法
    //严版教材的划分算法（一趟划分过程）
    ElemType pivot = A[low]; //将当前表中第一个元素设置为枢纽值，对表进行划分
    while(low < high)
    {
        while(low<high&&A[high]>=pivot)
            high--;
        A[low] = A[high];   //将比枢轴值小的元素移动到左端
        while(low<high&&A[low]<=pivot)
            low++;
        A[high] = A[low];  //将比枢轴值大的元素移动到右端
    }
    A[low] = pivot;  //枢轴元素存放到最终位置
    return low;   //返回存放枢轴的最终位置
}
```

重点掌握

- 性能分析
  1. 时间复杂度
     - **最坏情况下**：O(n²)
     - **平均情况下**：O(nlog₂n)
  2. 空间复杂度
     - **最好情况下**：O(log₂n)
     - **最坏情况下**：O(n)
     - **平均情况**：O(log₂n)
  3. 稳定性：**不稳定**

配套课程公众号[有机研]
永久联系微信:4550060

排序算法总结

内部排序算法的比较及应用

排序算法的比较

算法	时间复杂度			空间复杂度	稳定性	复杂性
	最好	平均	最坏			
直接插入	O(n)	$O(n^2)$	$O(n^2)$	O(1)	是	简单
冒泡	O(n)	$O(n^2)$	$O(n^2)$	O(1)	是	简单
选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	否	简单
希尔	-	O(nlogn) ~ $O(n^2)$	O(nlogn) ~ $O(n^2)$	O(1)	否	复杂
快速	O(nlogn)	O(nlogn)	$O(n^2)$	O(logn)	否	复杂
堆	O(nlogn)	O(nlogn)	O(nlogn)	O(1)	否	复杂
归并	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	是	复杂
基数	O(d(n+rd))	O(d(n+rd))	O(d(n+rd))	O(rd)	是	复杂

内部排序算法的应用

1. **看待排序元素数目n的大小**
   1. **n较小（n<=50）**
      n^2和$nlog_2n$的差别不大
      1. 直接插入排序
      2. 简单选择排序
   2. n较大 —— 采用时间复杂度为$O(log_2n)$的排序算法
      1. 快速排序
      2. 堆排序
      3. 归并排序
   3. n很大 —— 记录的关键字位数较少 —— 基数排序
2. **待排序序列基本有序**
   1. 直接插入 —— 比较次数更少，最常用，性能最佳
   2. 冒泡排序
3. **待排序的关键字随机分布** —— **快速排序**
   - 最好的排序方法
   - 一次循环就能确定一个元素的最终位置
4. **就辅助空间而言** —— 堆排序<快速排序<归并排序
5. **与初始状态有关** —— 交换类的排序 —— 冒泡排序
6. **与初始状态无关**
   1. 直接插入排序
   2. 简单选择排序
   3. 基数排序

配套课程公众号[有机研]
永久联系微信:4550060

外部排序算法

外部排序基本概念

- 如果参与排序的数据量特别大，存放在外村文件中，一次不能全部读入内存，用内排方法就不能完成对数据的整体排序
- 为此采用分段处理，每次将文件中一部分数据调到内存中进行排序，这样在排序过程中需要进行多次的内、外存之间的数据交换，这种排序称为外排序。

内存容量无法容纳大量的数据

多路归并排序

- 如何得到初始的归并段 —— 置换选择排序：解决排序段 放入内存的问题
- 如何减少多个归并段的归并次数 —— 最佳归并数：最少的归并次数
- 如何每次m路快速的最小关键字 —— 败者树：减少比较次数

置换选择排序

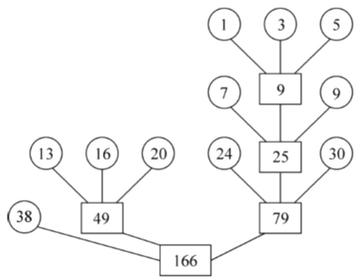
是在树形选择排序的基础上得来的， 它的特点是：在整个排序（得到所有初始归并段）的过程中， 选择最小（或最大）关键字和输入、 输出交叉或平行进行

最佳归并树

不同的归并方案所对应的归并树的带权路径长度各不相同，为了使总的读写次数达到最少，需要改变归并方案，重新组织归并树，使其路径长度WPL尽可能短。**所有归并树中最小带权路径长度WPL的归并树称为最佳归并树。**

最佳归并树是带权路径长度最短的k次（阶）哈夫曼树，构造过程如下：

1. 若(m-1)mod(k-1)≠0，则需附加(k-1)-(m-1)mod(k-1)个长度为0的**虚段**，以使每次归并都可以对应k个段。
2. 按照哈夫曼树的构造原则(权值越小的结点离根结点越远)构造最佳归并树。



一棵3路最佳归并树

此归并树的带权路径长度WPL=38×1+(13+16+20+24+30)×2+(7+9)×3+(1+3+5)×4=328，在归并过程中总的读写记录次数为2× WPL=656。

注：同样一组初始归并段，采用的归并路数也相同，但**选择不同的归并方案，读写记录次数可能是不相同的，但是最佳归并树对应的归并方案是读写记录次数最少的。**

多路平衡归并和败者树

- 败者树是一棵有k个叶子结点的完成二叉树（可将大根堆看成胜者树）
- 叶子结点存储的各个归并段当前参与归并的记录，分支结点存放关键字对应的段号，内部结点存储的则是左右子树当中的失败者
- 对k个有序段进行k路平衡归并的方法如下：
  1. 取每个输入有序段的第一个记录作为败者树的叶子结点，建立初始败者树：两两叶子结点进行比较，在双亲结点中存放比较的败者(关键字较大者)，而让胜者去参加更高一层的比赛，如此在根结点之上胜出的“冠军”是关键字最小者。
  3. 将胜出的记录写至输出归并段，在对应的叶子结点处补充其输人有序段的下一个记录，若该有序段变空，则补充一个大关键字(比所有记录关键字都大，设为kmax)的虚记录。
  3. 调整败者树，选择新的关键字最小的记录：从补充记录的叶子结点向上和双亲结点的关键字比较,败者留在该双亲结点，胜者继续向上，直到树的根结点，最后将胜者放在根结点的双亲结点中。

配套课程公众号[有机研]
永久联系微信:4550060